Beta Testing Document

for

UniPool

Version 1.0

Prepared by

Group #17 — **Group Name: 10 Bit**

Ayush Jha	230258	ayushj23@iitk.ac.in
Khushi Khandelwal	230561	khushik23@iitk.ac.in
Pranjali Singh	220796	pranjalis22@iitk.ac.in
Mohd Sarim Ali	230657	sarimali23@iitk.ac.in
Raghav Madan	220853	raghavm22@iitk.ac.in
Rigved Koli	230855	brigved23@iitk.ac.in
Ronit Kumar	230875	ronitk23@iitk.ac.in
Rushabh Pandya	230732	rushabhbp23@iitk.ac.in
Sanket Agarwal	230920	sanketag23@iitk.ac.in
Tanmay Roy	231082	tanmayroy23@iitk.ac.in

Course: CS253

Mentor TA: *V Arvind*

Date: 17/04/2026

Contents II
Revisions II

1 Introduction 1

2 List of Bugs 2

3 Overall quality of the software 4

Appendix A - Group Log 5

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Ayush Jha Khushi Khandelwal Pranjali Singh Mohd Sarim Ali Raghav Madan Rigved Koli	First draft of Beta Testing Document	16/04/26

Version	Primary Author(s)	Description of Version	Date Completed
	Ronit Kumar Rushabh Pandya Sanket Agarwal Tanmay Roy		

1 Introduction

UniPool is a self-contained mobile application designed exclusively for the student community at IIT Kanpur to facilitate collaborative ride-sharing. The application serves as a digital platform that connects students who share common travel destinations, enabling them to coordinate shared transportation, reduce individual travel costs, and contribute to environmental sustainability.

The application operates as a standalone system that bridges the gap between students who have planned travel and those seeking rides to similar destinations. Unlike traditional ride-sharing platforms that connect drivers with passengers, UniPool specifically targets the student community scenario where students collectively book a cab and share the cost rather than individual students owning vehicles.

2 List of Reported Bugs

BUG 1:

Tested Feature: Ride Cancellation / Deletion Notification

Tester Name: Ronit Kumar, Mohd Sarim Ali

Testing Date: 12/04/26

Bug Details:
Users (poolers) who joined a ride are not notified when the ride is deleted or cancelled.

Failing Test Case(s):

1. Join a ride as a pooler → Driver deletes the ride → No notification received.
2. Pooler opens the app after deletion → No indication that ride no longer exists.

Bug Report Date: 12/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 15/04/26

Any other comment: Issue resolved successfully.

BUG 2:

Tested Feature: Maximum Poolers Limit in a Ride

Tester Name: Rigved Koli, Tanmay Roy

Testing Date: 12/04/26

Bug Details:
There was no restriction on the number of users (poolers) who could join a ride, leading to unrealistic and overcrowded ride scenarios.

Failing Test Case(s):

1. Create a ride → Multiple users keep joining → No upper limit enforced.
2. Ride exceeds reasonable capacity → System still allows new poolers to join.

Bug Report Date: 12/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 15/04/26

Any other comment:
Fix implemented by adding a cap on the number of poolers and displaying current pooler count per ride.

BUG 3:

Tested Feature: Tentative Ride Time Display

Tester Name: Sanket Agarwal, Khushi Khandelwal, Rushabh Pandya

Testing Date: 12/04/26

Bug Details:
Ride posts did not include any estimated or tentative ride time, making it difficult for users to plan or decide whether to join a ride.

Failing Test Case(s):

1. Create a ride without specifying time → Ride gets posted successfully.
2. Users view ride details → No information about expected departure time.

Bug Report Date: 12/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 15/04/26

Any other comment:
Fix implemented by making tentative ride time mandatory before posting a ride.

BUG 4:

Tested Feature: Ride Joining Logic (Leader Restrictions)

Tester Name: Pranjali Singh, Raghav Madan, Ayush Jha

Testing Date: 12/04/26

Bug Details:
The ride creator (leader) was able to join their own ride. This caused the same ride to appear in both “I am leading” and “I joined” sections, leading to duplication and confusion.

Failing Test Case(s):

1. Create a ride as leader → Go to find rides → Join the same ride.
2. Check dashboard → Ride appears in both leading and joined sections.

Bug Report Date: 12/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 15/04/26

Any other comment:
Fix implemented by preventing created rides from appearing in the find rides section for the leader.

3 Overall Quality of the Software

Overall, UniPool is a useful and relevant application idea, and the software succeeds in implementing the main workflow of campus ride-sharing in a reasonably simple interface. The app provides core functionality such as ride creation, ride discovery, profile management, and in-app chat. The concept is practical and suitable for the student community it targets.

1. How good is the user manual? Was it easy to install and run the software?

The user manual is detailed and generally helpful. It explains the intended workflows clearly and gives a good understanding of how different parts of the application are supposed to work. For a tester, the manual makes it relatively easy to understand the expected behavior of the software. However, actually installing and running the software was not entirely seamless, as some environment and platform-related issues created friction during testing. So while the manual is good in terms of feature explanation, the practical setup experience could still be improved.

2. How is the quality of the code? Is it well documented? Is the code modular? Are the variables and the function names meaningful?

The quality of the code is moderate to good. The codebase is organized into separate screens and modules for key functionalities such as authentication, ride management, chat, and profile handling, which reflects a reasonable level of modularity. Variable names, file names, and overall structure are mostly meaningful and readable. At the same time, the implementation reveals certain gaps in validation, handling of corner cases, and consistency between the documentation and the actual behavior of the software. These issues suggest that the project has a solid structure but still needs polishing to reach a more mature level of reliability.

3. Do you think that the software lacks some major features that may make it less acceptable to the user?

The software also appears to lack a few important features that would significantly affect user acceptance in a real deployment setting. For example, proper handling of ride cancellation visibility, realistic limits on ride participation, and robust account verification are important for trust and day-to-day usability. In a ride-sharing application, even small coordination gaps can create confusion for users, so these missing or weakly implemented aspects matter more than they would in a less interactive system.

4. Does the software not satisfy any major non-functional requirements?

In terms of non-functional requirements, the software partially satisfies expectations but still falls short in some important areas. Usability is decent overall, but some features do not provide enough clarity or feedback to users. Reliability is also a concern in cases where important actions, such as verification or ride

updates, are not communicated effectively. These issues do not make the software unusable, but they do reduce confidence in the system and may affect the overall user experience.

Overall satisfaction:

Overall, we are moderately satisfied with UniPool. The software has a clear purpose, a relevant target audience, and a reasonably good functional base. It demonstrates thoughtful effort and good potential, but it still requires improvement in robustness, validation, and user-facing reliability before it can be considered fully polished and dependable for regular use.

Appendix A - Group Log

Date	Timing	Venue	Work Done
April 5	20:00-21:00	RM Building	• Initial team meeting to discuss the testing responsibilities and assignments
April 8	21:00-22:00	Google Meet	• Online meeting to go through the shared SRS, user manual, and test documentation.
April 11	20:00-21:00	RM Building	• Collaborative session to finalize bugs and publish test results on GitHub.
April 13	16:00-17:00	CCD	• Team met to distribute work for preparing the beta testing report.
April 16	20:00-01:00	Google Meet	• Final meeting to compile and complete the formal testing report as a team.